
Modules

1. Basics
2. The module math
3. The module random
4. Scientific modules
5. The module pandas
6. The module matplotlib

Basics

- You can use more functions by using additional modules in Python, which are a form of function libraries
- We need to use the import operator to read in these modules or parts of these modules into our normal python script
- These modules are nothing more than Python scripts that contain a lot of function definitions
- Note that your scripts should not have the same name as modules, e.g. calendar, since the import operator would look for a file called calendar.py for example
 - The program would try to read in itself, which works poorly at best

- To see which commands are available for standard modules, that means for modules that are part of the standard python library, either look in the library reference (if you downloaded it) or go to <https://docs.python.org/3/library/>
- Modules that are not part of the standard python library, e.g. NumPy or pandas, often have their own websites that include a detailed documentation about the different functions they give you access to

- The biggest advantage of these modules is that they increase our global namespace by adding more functions to it
- These functions can be simple things like generating nicer print outs or complex things like Fourier Transformations and so on
- Aside of these functions some modules also come with specific datastructures that can help us running more complex calculations
- Last but not least some modules contain mathematical and scientific constants like Pi or the Avogadro-constant and are therefore essential in scientific calculations

- When installing Thonny some modules are pre installed, while others need to be installed once before you can use them
- You can do this under Tools > Manage packages:
 - Type in the name of the module you want to install and click on “Search on PyPi”
 - Now click on the name of the module you want to install
 - Last but not least click on install and wait until the installation is finished
- When working with other IDEs you either have to search for a package/environment manager for this IDE or use a terminal and type:

```
pip install modulename
```
- Of course you have to replace modulename with the name of the module

- Keep in mind that you need to install these modules only once
- Afterwards you have access to them and you can import them into your scripts
- Many modules we use here are already pre-installed in Thonny
- Therefore try to start my scripts and see whether the used imports work or not
- Only if the program is not able to run, install the corresponding module

- When working with modules we have to import them into our script to get access to their functions, datastructures and constants
- This import should be done at the beginning of your script before you do anything else and you should write one import line per module
- There are different ways to import these modules or only parts of them
- The first and easiest one is to import the whole module as it is:

```
import calendar
```

- Now you have access to all functions of the module calendar and can use them with the help of the point notation:

```
calendar.prcal(2023)
```

- Another way of importing modules is by using an alias for the name of your module, to shorten your code later on

```
import calendar as cld
```

- Using an import like this still gives you access to the whole module calendar and you still need to use the point notation to use a specific function, but now you can use the alias before the point instead of the whole name:

```
cld.prcal(2023)
```

- The third way of importing things from these modules is by importing only specific functions from them, to add them to our global namespace:

```
from calendar import prcal
```

- Now we can use the function similar to our built-in functions like print by simply using its name:

```
prcal(2023)
```

- Please be careful than using an import like this, because you can overwrite already defined functions by using an import like this
- That means if there is already a function called prcal in the standard Python installation we would now use the calendar version instead

- Most modules have some form of import that is an international convention
- When showing you the different modules later on I will always use the import that Python programmers or the developers of a module agreed upon, so you should stick to these imports
- Importing only specific functions should be done only if the corresponding documentation tells you so, otherwise stick to importing the whole module with one of the first two examples

- One last thing I want to make you aware of, is the spamming of your namespace by using an import as follows:

```
from calendar import *
```

- This method would import all functions of the module calendar into your script, so that you can use them without the point notation
- However as soon as you have multiples of these imports and the modules contain functions with the same name, you only would have access to the last one you imported, which can be very troublesome
- **NEVER USE AN IMPORT LIKE THIS!!!!!!**

- Write a program that asks the user to guess a specific number
- Generate a random number between 0 and 99, search the library to find a module and a function for this task
- Generate a while loop that asks for a guess as long as the user didn't guess right
 - ➔ This is where you have to define your input
- Print out hints if the user is lower or higher than the random number by using an if / elif / else statement
- Count the number of guesses the user needed and print them out after the user guessed correctly

- When working with modules you don't need to memorize all functions they give you access to, since this is simply not possible
- Instead it's important to keep the general functionality of these modules in mind, that means what is the general aim of a certain module
- Later on you can look up specific functions in the standard python library or the documentation of your module
- Therefore we will look only at some example functions for each module and not talk about all functions of each module

The module math

- Provides access to mathematical functions and constants
- Cannot be used with complex numbers
 - Use `cmath` instead
- The distinction between functions that can use complex numbers and those which don't is made since most users don't want to learn quite as much mathematics
 - Furthermore receiving an exception / error instead of a complex number can make the program more resilient
- All returned values are floats

- `math.exp(x)`
 - Returns e raised to the power x
- `math.log(x[, base])`
 - With one argument returns the natural logarithm (to the base e)
 - With two arguments returns the logarithm of the first argument to the given base (second argument)
- `math.log2(x)`
 - Returns the logarithm of x to the base of 2
- `math.log10(x)`
 - Returns the logarithm of x to the base of 10

- `math.pow(x, y)`
 - Returns x raised to the power y
- `math.sqrt(x)`
 - Returns the square root of x
- `math.pi`, `math.e`, `math.tau`
 - $\pi = 3.141592\dots$
 - $e = 2.718281\dots$
 - $\tau = 6.283185\dots$

- If you're using the math module you have to pay attention that the operations you want to run are mathematically defined
- Otherwise you run into errors, the so called VALUE ERRORS with the message:
 - math domain error
 - This means that you're outside of the definition range of your chosen math function
- For example you're trying to calculate the square root of a negative number
- That's why you should use math only in combination with if statements to make sure that you can run your planned calculation

- Write a program that defines two numbers (x and y) and calculates new values with the help of the math functions on the previous slides (exp, log, log2, log10, pow, sqrt)
- You can define the numbers simply on your own by using $x = 5$ and $y = 10$ for example
- Print out the results

The module random

- As you should already know this module implements Pseudo-random (computer generated based on mathematical algorithms) number generators
- You maybe already saw:

```
random.randint(a,b)
```

➔ random integer N such that $a \leq N \leq b$

- Other methods are:

```
random.random()
```

➔ random number between 0 and 1

```
random.gauss(mu,sigma)
```

➔ normal distribution

```
random.shuffle(list1)
```

➔ mixes list1 randomly

```
random.randrange(a,b)
```

➔ integer N such that $a \leq N < b$

- Write a program that generates several random numbers.
- Define the following:
 - ▶ three random integers stemming from different ranges
 - ▶ a random floating point number between 0 and 1
 - ▶ two values from normal distributions using different mu and sigma values
- What happens if you switch these last values?
- Last but not least put all generated values in a list, print the list, shuffle the list and print it again
 - ▶ Just use `list1 = [ ]` and write the first 6 results in form of variables names inside the square brackets

Scientific modules

- The fundamental package for scientific computing in Python, since it gives us access to more advanced mathematical functions as well as a datastructure, the ndarray, that can be used for these calculations
- The scientific module SciPy which gives us access to even more scientific functions is based upon NumPy and the ndarray
- `import numpy as np` ➔ convention for the import of the numpy module

- At the core of the NumPy package, is the ndarray object, a new datastructure that can be used to model vectors and matrices in Python
- NumPy arrays have a fixed size at creation, that means we can NOT add or remove elements from them
- The elements in a NumPy ndarray are all required to be of the same data type, that means all elements have to be floats or integers
- In general we can cast lists, sets and tuples to numpy ndarray by using the function `np.array()`:

```
np.array([4, 3, 2, 1])
```

- The first and most general way of creating an ndarray is by casting a list, set or tuple to an array by using `np.array()`:

`np.array([4, 3, 2, 1])` → `[4 3 2 1]`

- NumPy also has a function `np.arange` that creates an array with integers or floats, it follows the same rules as the built-in `range()` function when it comes to the arguments of the function:

`np.arange(4)` → `[0 1 2 3]`

- We also can create arrays that contain only zeros with the function `np.zeros()`, but these are floating point numbers:

`np.zeros(4)` → `[0. 0. 0. 0.]`

- Furthermore we also have a function that creates an array consisting only of floating point versions of the number 1, by using `np.ones()`:
 - ▶ `np.ones(4)` → `[1. 1. 1. 1.]`
- We can create multi-dimensional arrays by using `np.zeros()` or `np.ones()` with a tuple of arguments. In the example below I create an array consisting of three rows and each row contains four elements:
 - ▶ `np.zeros((3,4))` → `[[0. 0. 0. 0.]`
`[0. 0. 0. 0.]`
`[0. 0. 0. 0.]]`

- We can also generate arrays of specific sizes containing random numbers by using the `np.random.randn()` function
 - ▶ `np.random.randn(6, 4)` → ?
- These numbers are taken from a “standard normal distribution” that means a normal distribution with a mean of 0 and a standard deviation of 1 [7]

- All operations on arrays apply elementwise:
 - ▶ `[1 2 3] * 3` → `[3 6 9]`
 - ▶ `[10 20 30 40] - [0 1 2 3]` → `[10 19 28 37]`
- There are basic functions like:
 - ▶ `array.sum()`
 - ▶ `array.min()`
 - ▶ `array.max()`
 - ▶ `array.transpose()`
 - ▶ `array.shape`

- Write a simple python program that generates the following arrays:
 - ▶ An array consisting of 25 zeros
 - ▶ An array consisting of 8 ones
 - ▶ An array consisting of numbers from 0 to 24 in order
 - ▶ An array consisting of 9 entries of your choice
 - ▶ An two-dimensional array “TD” consisting of 6 rows of arrays with 5 entries each
 - ▶ A transposed version of the two-dimensional array “TD”
- Furthermore print out the sum, maximum value, minimum value and shape of all created arrays

- NumPy gives us access to a wide number of different mathematical operations we can run on our arrays
- Each of these methods work elementwise
 - `np.sin`, `np.cos`, `np.tan`, `np.arcsin`, `np.sinh` etc...
 - `np.log`, `np.log10`, `np.log2`, `np.exp` etc...
 - `np.sqrt`, `np.power` etc...
- There are also methods for linear algebra `np.linalg` or for Fourier Transformations `np.fft`

- Use the arrays from Exercise 57 and run some mathematical functions like `np.sin` with them
- You can just add the calculations to your script from exercise 57
- Don't forget to print out the results

- SciPy is a collection of mathematical algorithms and convenience functions built on the NumPy extension of Python
- With SciPy, a python script becomes we can run data-processing operations just like we could in systems, such as MATLAB, IDL, Octave, R-Lab and SciLab
- SciPy sub-packages need to be imported separately:
 - `from scipy import linalg, optimize`
- And then we can use functions from them using the point notation:
 - `linalg.inv(matrix)`

Subpackage	Description
cluster	Clustering algorithms
constants	Physical and mathematical constants
fftpack	Fast Fourier Transform routines
integrate	Integration and ordinary differential equation solvers
interpolate	Interpolation and smoothing splines
io	Input and Output
linalg	Linear algebra
ndimage	N-dimensional image processing
odr	Orthogonal distance regression
optimize	Optimization and root-finding routines
signal	Signal processing
sparse	Sparse matrices and associated routines
spatial	Spatial data structures and algorithms
special	Special functions
stats	Statistical distributions and functions

The module pandas

- When working with tabular data, such as data stored in spreadsheets or databases, pandas is the right tool for you
- In pandas, a data table is called a DataFrame
- pandas supports the integration with many file formats or data sources out of the box
 - csv, excel, sql, json, parquet
- There is no need to loop over all rows of your data table to do calculations
 - Data manipulations on a column work elementwise
 - Adding a column to a DataFrame based on existing data in other columns is straightforward

- Standard import:

```
import pandas as pd
```

- Two data structure types:

- ▶ Series (one-dimensional):

```
s = pd.Series()
```

- ▶ DataFrame (two-dimensional):

```
df = pd.DataFrame()
```

- A Series is a one-dimensional datastructure similar to lists or tuples
- However pandas interprets them as a singular column and thus we can use pandas functions on it to run calculations over it
- We create a Series using the `Series()` function of pandas
- This function doesn't need any arguments:

```
series1 = pd.Series()
```

- When used like this we create an empty Series
- However we can also hand over values in form of an iterable to it to create a Series with data:

```
series2 = pd.Series([1, 3, 5, np.nan, 6, 8])
```


- Last but not least we can also specify the optional argument `index` in combination with an iterable to define row names for this datastructure

```
series3 = pd.Series([1, 3, 5, np.nan, 6, 8], index=["A", "B", "C", "D", "E", "F"])
```

- The row names (here called `index`) can later be used to access the data in our Series
- Please keep in mind that the amount of values must be identical to the amount of row names

- For two-dimensional data we can use the datastructure DataFrame of the module pandas
- This datastructure allows us to work with tabular data inside our scripts in very simple manner
- The general definition is as follows and we can of course create empty DataFrames by using it with out arguments:

```
df1 = pd.DataFrame()
```

- We can also create a DataFrame out of consisting data, e.g. with the help of the `random()` function of numpy:

```
df2 = pd.DataFrame(np.random.randn(2, 4))
```

- The DataFrame `df2` now contains two rows and four columns of data
- Just like we saw for the Series we can also name the rows of a DataFrame with the help of the optional argument `index`:

```
df3 = pd.DataFrame(np.random.randn(2, 4), index=["I1", "I2"])
```

- Furthermore we can name the columns of our DataFrame with the help of the optional argument `columns`:

```
df4 = pd.DataFrame(np.random.randn(2, 4), columns=["A", "B", "C", "D"])
```

- We can combine these two optional arguments as well to name the rows and the columns of our DataFrame at the same time:

```
df5 = pd.DataFrame(np.random.randn(2, 4), index=["I1", "I2"],  
columns=["A", "B", "C", "D"])
```

- One nice thing about defining row and column names is, that it allows you to create a DataFrame without data as well:

```
df6 = pd.DataFrame(index=["I1", "I2"], columns=["A", "B", "C", "D"])
```

- Write a program that creates multiple DataFrames
- The DataFrames should have the following dimensions:
 - ▶ 6 rows and 4 columns
 - ▶ 4 rows and 8 columns
 - ▶ 10 rows and 3 columns
 - ▶ 5 rows and 9 columns

- Just like lists, DataFrames also have access to a variety of different methods
- Here I will go over some of the most important ones
- In general these methods work column wise, that means we get one result for each column in our DataFrame
- If you want to use them row wise instead you have to add the argument `axis=1` to the call of the method

- First we are able to calculate the sum of all values in the columns of our DataFrame:

```
df1.sum()
```

- For the sums of the different row you would need to use the following instead:

```
df1.sum(axis=1)
```

- A second useful method is `describe()`:
- This method produces a description of our DataFrame by showing us the mean values, standard deviations and other information about each column of our DataFrame:

```
df1.describe()
```


- We can sort our DataFrame according to the row names by using the method `sort_index()`
- This is one of the exceptions, since this function works row wise per default instead of column wise:

```
df1.sort_index()
```

- The sorting is done in ascending order and if you want the sorting to be in descending order you need to add the argument `ascending=False` to it:

```
df1.sort_index(ascending=False)
```

- To sort according to column names you need to add `axis=1`:

```
df1.sort_index(axis=1, ascending=False)
```

- We can sort the data in our DataFrame according to the values in one column by using the method `sort_values()`
- Here we need to specify the column name(s) as a list of our choice:

```
df1.sort_values(["B"])
```

- If you want to sort according to multiple columns you can hand over the names of them in form of a list to the method:

```
df1.sort_values(["A", "B"])
```

- Of course you can also sort according to the values of a row by using the argument `axis=1` and specifying a row name instead:

```
df1.sort_values(["Test1"], axis=1)
```

- Last but not least we can get access to all row names with the help of the attribute index:

```
df1.index
```

- In a similar manner we can get access to all column names with the help of the attribute columns:

```
df1.columns
```

- On the last slide you saw that DataFrames have the so-called attributes index and columns
- Attributes are similar to methods, since they're used on an object of a specific type, here a DataFrame
- The main difference is that they're used without bracket, that means we can't hand over any arguments to them
- This results from the fact, that attributes are variables of the corresponding object and not functions
- You will learn more about this in the object oriented part later on

- Use your DataFrames from Exercise 59 to fulfill the following tasks:
 - ▶ Calculate the columnwise sums for the first DataFrame
 - ▶ Calculate the rowwise sums for the second DataFrame
 - ▶ Sort the fourth DataFrame according to the values in the first and third column
 - ▶ Create a description of the third DataFrame
 - ▶ Print out the row and column names of the second DataFrame

- If you want to access elements in DataFrames you have two different methods

1. You can use Indices with `.iloc`

```
df1.iloc[3, 2]
```

➔ gives you access to the element in the fourth row and the third column (indexing starts at 0)

2. You can use row and column names with `.loc`

```
df1.loc["Test4", "C"]
```

➔ gives you access to the element in the row "Test4" and the column "C"

- If you want to access complete columns and row you can use the same methods we saw before (.loc or .iloc)
- Replace the column name/index with a : if you want to access a complete row
- Replace the row name/index with a : if you want to access a complete column

- Use your DataFrames from Exercise 59 to fulfill the following tasks:
 - ▶ Print out the element in the first column and first row of DataFrame1
 - ▶ Print out the element in the third column and the second row of DataFrame2
 - ▶ Print out the fifth row of DataFrame3
 - ▶ Print out the sixth column of DataFrame4
- Use both versions (loc and iloc to fulfill the tasks)

- I mentioned that we can add new columns or rows to our DataFrames in a simple manner
- This can be done with the help of the `.loc` method and the name of a nonexisting column/row
- The new column/row is then just added to the end of the DataFrame:

```
df1.loc["Test2", "K"] = 25
```

```
df1.loc["Test5", "D"] = 33
```

- Use your DataFrames from Exercise 59 to fulfill the following tasks:
 - ▶ Place a new element in the fifth row of a new column into DataFrame1
 - ▶ Place a new element in the fourth column of a new row into DataFrame2
 - ▶ Place a new element in the seventh row of a new column into DataFrame3
 - ▶ Place a new element into a new row and new column into DataFrame4

- Pandas allows us to read in multiple different files
- There are methods for reading in csv-files (`read_csv`), Excel-files (`read_excel`), XML-files (`read_xml`), JSON-files (`read_json`) and many more
- You can also save your DataFrames in the different formats with methods like `to_xml`, `to_csv`, `to_excel` and so on
- If you want to work with excel-files you need to install the module `openpyxl` in addition to `pandas`

- Use your DataFrame from Exercise 59 and save it to a csv-file and an Excel-file
- Try to read these files with the help of pandas afterwards

The module Matplotlib

- A comprehensive library for creating static, animated, and interactive visualizations in Python
- Allows you to:
 - ▶ Create publication quality plots
 - ▶ Make interactive figures that can zoom, pan, update
 - ▶ Customize visual style and layout
 - ▶ Export to many file formats
 - ▶ Embed in JupyterLab and Graphical User Interfaces
 - ▶ Use a rich array of third-party packages built on Matplotlib

- To make use of matplotlib you need to import its submodule pyplot
- The import is as follows:

```
import matplotlib.pyplot as plt
```

- As you can see we use an alias again when working with this module

- Matplotlib offers a lot of different plot types:
 1. Scatter- or Line-plots using `plt.plot()`
 2. Vertical Bar-plots using `plt.bar()`
 3. Scatter- or Line-plots with Errors using `plt.errorbar()`
 4. Horizontal Bar-plots using `plt.barh()`
 5. Histograms using `plt.hist()`
 6. Contour-plots using `plt.contour()`
 7. Heatmaps using `plt.imshow()`
 8. Pie-charts using `plt.pie()`
 9. 3D-Surface-plots with `plot_surface()`
 10. 3D-Bar-plots with `bar3d()`

- You can modify your plots by:
 - ▶ Adding a diagram title
 - ▶ Adding labels for the axes
 - ▶ Specifying labels for the interval ticks of the axes
 - ▶ Specifying the interval ticks of the axes
 - ▶ Defining the ranges of the axes
 - ▶ Adding text to your diagram
 - ▶ Adding a grid to your diagram
 - ▶ Specifying the colors and markers for your data points
 - ▶ Specifying the type of line you want to draw
 - ▶ ...

- To create a plot with matplotlib you need to create a canvas with the help of the function `figure()` first:

```
plt.figure()
```

- Now you can draw your diagram on this canvas, for example a simple line-plot using the function `plt.plot()`:

```
plt.plot(xvalues, yvalues)
```

- The two arguments `xvalues` and `yvalues` need to be two datastructures with the same amount of elements
- Keep in mind that these elements need to be numerical (int or float)

- Now you can look at your plot with the function `plt.show()`:

```
plt.show()
```

- Once you're done you need to close the canvas with `plt.close()`

```
plt.close()
```

- At the end your script looks as follows:

```
import matplotlib.pyplot as plt
```

```
xvalues = [1,2,3,4]
```

```
yvalues = [10,12,9,11]
```

```
plt.figure()
```

```
plt.plot(xvalues,yvalues)
```

```
plt.show()
```

```
plt.close()
```

- Instead of showing your plot you can also save your plot by using the `savefig()` function:

```
plt.savefig('Image.png')
```

- Matplotlib supports png, tiff, pdf, svg among other datatypes with its `savefig()` function
- Be aware that you can only look at your plot using `show()` or save the plot using `savefig()` but not both

- In most cases you want to add labels, titles and so on to your plot, that means you want to modify it
- Nearly each and every aspect of a plot created with matplotlib is adjustable and here you will see some of the more common adjustments

- To add a title to your diagram you can use the function `title()`:

```
plt.title(„Lineplot“)
```

- If you want to add a small gap between the title and your plot you can use the optional argument `y`:

```
plt.title(„Lineplot“, y=1.02)
```

- To add labels to your axes you can use the functions `xlabel()` and `ylabel()`:

```
plt.xlabel („Xvalues“)
```

```
plt.ylabel („Yvalues“)
```

- Please keep in mind that the argument for these functions needs to be a string

- You can specify the borders / limits of your axes with the functions `xlim()` and `ylim()`:

```
plt.xlim([-1, 1])
```

```
plt.ylim([-1, 1])
```

- These functions take lists as arguments and these lists need to contain two elements, the start (-1) and end point (1) of your axes limits
- You're free to choose the values here and you can even use something as follows:

```
plt.xlim([1, -1])
```

- You can specify the intervals on your axes using the functions `xticks()` and `yticks()`:

```
plt.xticks([-1, -0.5, 0, 0.5, 1])
```

```
plt.yticks(np.arange(-1, 1.2, 0.2))
```

- You can even use customized interval labels:

```
plt.xticks(xvalues, names)
```

- In this case `xvalues` is a datastructure that specifies the intervals and `names` specifies the interval labels
- Both datastructures need to contain the same amount of elements

- You can add grid lines to your plot with the function `grid()`:

```
plt.grid()
```

- If you want to draw the grid lines only for one of your axes you can use the optional argument `axis`:

```
plt.grid(axis='y')
```

```
plt.grid(axis='x')
```

- You can also add a legend to your plot, if you have multiple lines for example, with the function `legend()`

```
plt.legend(('power 1','power 2','power 3','power 4','power 5'))
```

- This function takes a tuple as argument and this tuple needs to contain one element for each line in your plot
- These elements need to be strings so that they can be displayed in your legend

- string – for dealing with strings
- re – for regular expressions and better string method
- datetime – when dealing with Dates and times
- glob – for searching for files on your harddrive
- pickle – to save variables as files on your harddrive
- os – to move through the directories on your harddrive
- tkinter – for generating graphical user interfaces

- If you want more information you can find:
 - ▶ Tutorials: <https://matplotlib.org/stable/tutorials/index>
 - ▶ Examples: <https://matplotlib.org/stable/gallery/index>
 - ▶ A User Guide: <https://matplotlib.org/stable/users/index.html>
 - ▶ The reference: <https://matplotlib.org/stable/api/index.html>
- On the webpage of matplotlib

- Create three different plots with the help of Matplotlib:
 - ▶ a Lineplot with at least 10 datapoints
 - ▶ a Scatterplot with at least 500 datapoints
 - ▶ a bar diagram with at least 15 bars
- Format your plots accordingly by:
 - ▶ adding title, Axes labels
 - ▶ specifying Limits, intervals and interval labels for both Axes
- Save your plots as png images

1. https://en.wikibooks.org/wiki/Non-Programmer%27s_Tutorial_for_Python_3 (Last edited: 05.09.2022, Last visited: 02.01.2023)
2. <https://realpython.com/python-import/> (Last visited: 03.01.2023)
3. <https://docs.python.org/3/library/os.html> (Last edited: 03.01.2023, Last visited: 03.01.2023)
4. <https://docs.python.org/3/library/math.html> (Last edited: 03.01.2023, Last visited: 03.01.2023)
5. <https://docs.python.org/3/library/random.html> (Last edited: 03.01.2023, Last visited: 03.01.2023)
6. NumPy User Guide, Release 1.22.0 (Last edited: 14.1.2022, Download: 27.1.2022)
7. https://sphweb.bumc.bu.edu/otlt/mph-modules/bs/bs704_probability/bs704_probability9.html (Last edited: 24.06.2016, Last visited: 11.04.2023)
8. <https://scipy.github.io/devdocs/index.html> (Last edited: 02.01.2023, Last visited: 03.01.2023)
9. pandas: powerful Python data analysis toolkit, Release: 1.4.0 (Last edited: 22.1.2022, Download: 27.1.2022)
10. <https://matplotlib.org> (Last visited: 27.09.2023)